Builder

Season: 1 · Creational · Category: GoF · Level: Beginner

Intent

- Construct complex immutable objects step by step.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- A sandwich order form: bread, fillings, sauces — then 'build'.

When to Use

- Many optional parameters; telescoping constructors hurt.
- The design force named in Intent shows up more than once.

When Not to Use

- Two-field records — just use the canonical constructor.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.gof.creational.builder;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Objects;

/** Immutable HTTP request built with a fluent classic builder. */
public final class HttpRequest {
    private final String method;
    private final String url;
    private final Map<String, String> headers;
    private final String body;
    private final int timeoutMs;

    private HttpRequest(Builder b) {
        this.method = b.method;
        this.url = b.url;
        this.headers = Map.copyOf(b.headers);
        this.body = b.body;
        this.timeoutMs = b.timeoutMs;
    }

    public String method() { return method; }
    public String url() { return url; }
    public Map<String, String> headers() { return headers; }
    public String body() { return body; }
    public int timeoutMs() { return timeoutMs; }
```

Classic Implementation (2)

```
    public static Builder builder(String url) { return new Builder(url); }

    /** Modern: records + withers-style nested builder still works well. */
    public static final class Builder {
        private String method = "GET";
        private final String url;
        private final Map<String, String> headers = new LinkedHashMap<>();
        private String body = "";
        private int timeoutMs = 5_000;

        private Builder(String url) {
            this.url = Objects.requireNonNull(url);
        }

        public Builder method(String method) { this.method = method; return this; }
        public Builder header(String k, String v) { headers.put(k, v); return this; }
        public Builder body(String body) { this.body = body; return this; }
        public Builder timeoutMs(int timeoutMs) { this.timeoutMs = timeoutMs; return this; }

        public HttpRequest build() {
            if (timeoutMs <= 0) throw new IllegalArgumentException("timeoutMs must be > 0");
            return new HttpRequest(this);
        }
    }
}

/** Java 25-friendly: compact record with smart constructor + defaults via static factory. */
record ApiCall(String method, String url, String body, int timeoutMs) {
    ApiCall {
        Objects.requireNonNull(method);
        Objects.requireNonNull(url);
        if (timeoutMs <= 0) throw new IllegalArgumentException("timeout");
        body = body == null ? "" : body;
    }
```

Classic Implementation (3)

```
    static ApiCall get(String url) { return new ApiCall("GET", url, "", 5_000); }
    ApiCall withBody(String body) { return new ApiCall(method, url, body, timeoutMs); }
    ApiCall withTimeout(int ms) { return new ApiCall(method, url, body, ms); }
}
```

Modern Java 25 / Functional

- Records with withers (ApiCall.withBody) for lighter builders.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: gof/creational/builder/
```

Pitfalls

- Validate in build().
- Prefer immutable results (Map.copyOf).

Interview Q&A

- Q: Builder vs telescoping constructors? A: Builder scales; telescoping explodes combinations and invites arg-swap bugs.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Construct complex immutable objects step by step.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: A sandwich order form: bread, fillings, sauces — then 'build'. [0:40] Intent — State the design forc...